

Nifty 100 Data Foundation

Analyst Guide

1. Introduction

The Nifty 100 Data Foundation project is a financial data analysis platform designed to provide structured financial information, company-level analysis, screening, peer comparison, sector analysis, valuation information, cash-flow intelligence and generated reports for Nifty 100 companies.

The project combines a structured SQLite database, financial analysis modules, a Streamlit dashboard, a FastAPI REST API and automated PDF reporting.

The current implementation contains financial information for **92 sector-mapped companies across 10 sectors**. The dashboard provides multiple views so that an analyst can move from an overall market view to individual company analysis.

The main purpose of the project is to make financial information easier to access and analyse without manually working through multiple spreadsheets and reports.

The major components of the project are:

- Data ingestion and ETL
- Data validation and quality checks
- Financial ratio calculations
- CAGR and growth analysis
- Screener and peer comparison
- Sector analysis
- Valuation analysis
- Cash-flow intelligence
- NLP-based pros and cons
- Company clustering
- Streamlit dashboard
- FastAPI REST API
- Company and sector PDF reports

The project is organised so that the same underlying financial data can be used through the dashboard, API and generated reports.

2. Project Structure

Nifty100_DataFoundation/

```
|
|
|— config/
|   └─ screener_config.yaml
|
|
|— data/
|   └─ nifty100.db
|       └─ raw/
|           └─ core/
|               └─ supporting/
|
|
|— docs/
|   └─ openapi.json
|   └─ nifty100_postman_collection.json
|   └─ verification documents
|   └─ sprint retrospectives
|       └─ analyst_guide.pdf
|
|
|— notebooks/
|   └─ exploratory_queries.sql
|
|
|— output/
|   └─ NLP outputs
|   └─ cash-flow intelligence
|   └─ clustering outputs
|   └─ screener and valuation outputs
|   └─ QA and validation outputs
```

```
|   └─ final_deliverables/
|
|   └─ reports/
|       └─ portfolio/
|       └─ radar_charts/
|       └─ sector/
|       └─ tearsheets/
|       └─ correlation_heatmap.png
|       └─ elbow_plot.png
|       └─ pytest_report.html
|
|   └─ src/
|       └─ analytics/
|           └─ CAGR analysis
|           └─ financial ratios
|           └─ cash-flow intelligence
|           └─ capital allocation
|           └─ clustering
|           └─ peer analysis
|           └─ quality scoring
|           └─ screener
|           └─ valuation
|           |
|           └─ api/
|               └─ main.py
|               └─ routers/
|               |
|               └─ dashboard/
```

```
| | └─ app.py
| | └─ pages/
| |   └─ 01_home.py
| |   └─ 02_profile.py
| |   └─ 03_screener.py
| |   └─ 04_peers.py
| |   └─ 05_trends.py
| |   └─ 06_sectors.py
| |   └─ 07_capital.py
| |   └─ 08_reports.py
| | └─ utils/
| |
| └─ db/
|   └─ schema.sql
|   └─ database utilities
|
| └─ etl/
|   └─ normaliser.py
|   └─ validator.py
|   └─ loader.py
|
| └─ nlp/
|   └─ parser.py
|   └─ cagr_validation.py
|   └─ pros_cons_generator.py
|
| └─ reports/
|   └─ tearsheet.py
```

```
| | | └─ batch_reports.py
| | | └─ sector_reports.py
| | └─ portfolio_summary.py
| |
| └─ screener/
|   └─ engine.py
|   └─ presets.py
|   └─ composite_score.py
|   └─ export.py
|
└─ tests/
    └─ api/
    └─ dq/
    └─ etl/
    └─ kpi/
    └─ performance/
|
└─ README.md
└─ requirements.txt
└─ pytest.ini
└─ Makefile
```

3. System Requirements

The project requires Python and the project dependencies installed in the virtual environment.

Recommended setup:

- Windows operating system
- Python 3.x
- PowerShell or Command Prompt
- Virtual environment

- SQLite database
- Internet connection when external data or package installation is required

The project should be executed from the project root directory.

Example: " **D:\Nifty100_DataFoundation** "

Activate the virtual environment before running project commands.

Example: " **.\venv\Scripts\Activate.ps1** "

After activation, the PowerShell prompt should indicate that the virtual environment is active.

4. Running the Data Pipeline

The ETL process is responsible for preparing financial data and loading it into the project's SQLite database. The general data pipeline consists of:

1. Reading source financial data
2. Normalising the data
3. Validating the data
4. Loading the cleaned data into SQLite
5. Running financial calculations
6. Generating analysis outputs

Data-quality checks are used to identify invalid or incomplete records before the information is used by the analysis modules.

The database used by the project is : " **data/nifty100.db** "

Important generated data-quality files include:

output/load_audit.csv

output/validation_failures.csv

The validation output can be reviewed when checking whether the source data was loaded correctly.

The ETL scripts are located under : " **src/etl/** "

The main ETL components include the normaliser, validator and loader.

5. Starting the Streamlit Dashboard

The Streamlit dashboard provides the main user interface for analysing the financial data.

From the project root, set the Python path and start Streamlit:

```
$env:PYTHONPATH="$PWD\src"  
streamlit run src/dashboard/app.py --server.port 8501
```

After starting successfully, open the dashboard in a browser using : " <http://127.0.0.1:8501> "

The dashboard is designed for interactive analysis. Analysts can select companies, sectors and screening criteria depending on the selected screen.

The dashboard contains eight main screens:

1. Home
2. Company Profile
3. Screener
4. Peer Comparison
5. Trends
6. Sectors
7. Capital Allocation
8. Reports

6. Home Screen

The Home screen provides the starting point for the dashboard.

It gives the analyst an overall view of the available Nifty 100 financial analysis platform and provides access to the other sections.

The Home screen can be used to understand the purpose of the application before moving into detailed analysis.

From the Home screen, the analyst can navigate to company-level analysis, screening, peer comparison, sector analysis, capital allocation and reports.

The Home screen is mainly intended as an entry point rather than a detailed analysis screen.

7. Company Profile Screen

The Company Profile screen provides detailed information about an individual company.

The analyst selects a company or ticker and can review the available financial information. The screen provides information such as:

- Company name
- Ticker
- Revenue
- Net profit
- Financial ratios
- ROE
- ROCE
- Debt-to-equity
- Profitability information
- Financial trends
- Cash-flow information
- Valuation information

The screen also presents financial charts to make changes over multiple years easier to understand.

When reviewing a company, the analyst should first look at the key performance indicators and then review the historical trends. The Company Profile screen is useful when detailed information is required for one company.

8. Screener Screen

The Screener allows an analyst to filter companies based on financial characteristics. The screener is useful when the analyst does not want to manually inspect every company.

Typical financial measures available through the project include profitability, growth, leverage and quality-related metrics.

The analyst can select the required screening conditions and review the companies that satisfy the selected criteria.

The screener results are displayed in a structured table. The screener can also be used to identify a smaller group of companies for further investigation.

The analyst should treat screening as a filtering mechanism. A company appearing in the screener results does not replace detailed financial analysis.

9. How to Use the Streamlit Screener

To use the Screener:

Step 1 — Open the dashboard

Start Streamlit using:

```
$env:PYTHONPATH="$PWD\src"  
streamlit run src/dashboard/app.py --server.port 8501
```

Open:

" <http://127.0.0.1:8501> "

Step 2 — Open Screener

Select the **Screener** screen from the dashboard navigation.

Step 3 — Select screening conditions

Choose the required filters and financial conditions.

The filters can be used to narrow the company universe according to the selected financial criteria.

Step 4 — Review the results

The resulting companies are displayed in a table.

Review the available metrics and compare the companies against the selected criteria.

Step 5 — Export the results

The screener provides an option to export the filtered results.

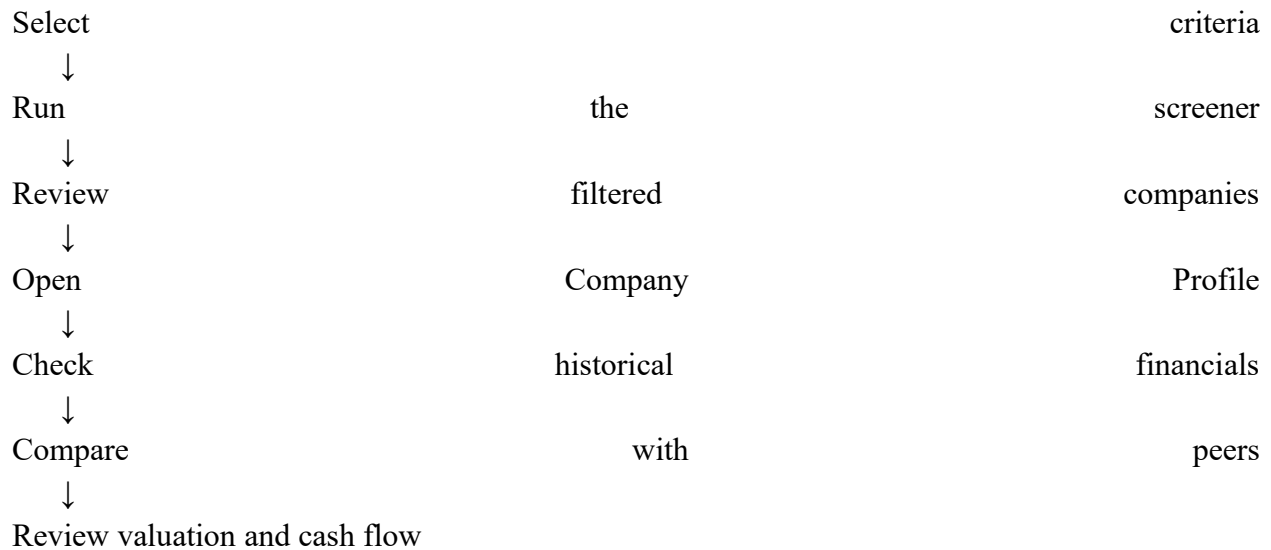
The project also maintains the generated screener output:

output/screener_output.xlsx

The exported results can be used for further analysis outside the dashboard.

Recommended workflow

A practical workflow is:



This allows the screener to be used as the first step in a broader analysis rather than as the final decision point.

10. Peer Comparison Screen

The Peer Comparison screen is used to compare companies within a relevant peer group.

The analyst can select a company or peer group and review comparable financial metrics.

Peer analysis can help identify differences in:

- Revenue growth
- Profitability
- ROE
- Leverage
- Other financial ratios

The project also contains peer comparison output : **output/peer_comparison.xlsx**

The analyst should consider the business context when interpreting peer comparisons because companies may have different business models, capital structures and operating characteristics.

11. Trends Screen

The Trends screen provides historical financial information over multiple years.

It is useful for identifying changes in financial performance rather than looking only at the latest year. The analyst can review trends such as:

- Revenue
- Net profit
- Profitability
- ROE
- ROCE
- Other available financial metrics

Trend analysis is particularly useful for understanding whether a company's performance has changed consistently over time. Historical information should be interpreted together with the available financial statements and ratios.

12. Sectors Screen

The Sectors screen provides a sector-level view of the companies in the project.

The current implementation contains **10 broad sectors**:

1. Communication Services
2. Consumer Discretionary
3. Consumer Staples
4. Energy
5. Financials
6. Healthcare
7. Industrials
8. Information Technology
9. Materials
10. Real Estate

The analyst can select a sector and review the companies associated with that sector.

Sector analysis is useful for comparing companies operating in similar business environments.

The project also generates sector reports under : " **reports/sector/** "

Each generated sector report contains sector-level analysis based on the available project data.

13. Capital Allocation Screen

The Capital Allocation screen provides information about how companies allocate capital.

The project classifies capital allocation patterns using financial information available in the database.

The generated capital allocation file is : `output/capital_allocation.csv`

The analysis can be used to understand patterns related to areas such as:

- Dividends
- Debt
- Investments
- Asset-related activity
- Other capital allocation behaviour

The analyst should review capital allocation together with profitability and cash-flow information because capital allocation cannot be interpreted independently from the company's financial position.

14. Reports Screen

The Reports screen provides access to generated reports and company-level reporting information.

The project generates company tearsheets and sector reports.

Company tearsheets are stored under : "**reports/tearsheets/**"

Sector reports are stored under : "**reports/sector/**"

The portfolio summary is stored under : "**reports/portfolio/**"

The Reports screen is useful when the analyst needs a report-based view instead of interactive dashboard analysis.

15. Generating Company PDF Tearsheets

A company tearsheet is a two-page A4 PDF containing a summary of important company information. The tearsheet includes information such as:

- Company name and ticker

- Key performance indicators
- Revenue
- Net profit
- ROE
- ROCE
- Debt-to-equity
- P/E
- Revenue and profit charts
- ROE/ROCE trends
- Balance-sheet information
- Cash-flow information
- Pros and cons
- Capital allocation information

Generated company tearsheets are stored in : " **reports/tearsheets/** "

The project includes a tearsheet generation module : " **src/reports/tearsheet.py** "

Company tearsheets can be generated using : " **python -m src.reports.tearsheet** "

A batch reporting module is also available for generating reports for multiple companies.

After generating reports, the PDF files should be checked for:

- Correct company name
- Correct ticker
- Correct KPI values
- Correct charts
- Correct pros and cons
- Correct capital allocation information
- Proper page layout
- No text overflow
- No clipped content

The five important sample companies used for visual verification include:

- TCS
- HDFCBANK
- RELIANCE
- SUNPHARMA
- TATASTEEL

16. FastAPI Overview

The project provides a REST API using FastAPI. The API allows applications or analysts to retrieve financial information programmatically.

Start the API from the project root using the configured FastAPI application.

The API runs on : " **http://127.0.0.1:8000** "

The API health endpoint is : " **/api/v1/health** "

The API provides endpoints for:

- Companies
- Profit and loss
- Balance sheet
- Cash flow
- Financial ratios
- Tearsheets
- Screener
- Sectors
- Sector companies
- Peer groups
- Peer comparison
- Market capitalisation
- Portfolio statistics
- Company documents

The OpenAPI specification is available at : " **docs/openapi.json** "

A Postman collection is also available at : " **docs/nifty100_postman_collection.json** "

17. API Examples Using curl

The following examples assume that the FastAPI server is running on port 8000.

Check API health

" curl **http://127.0.0.1:8000/api/v1/health** "

A successful response should contain a status indicating that the API is healthy.

Get companies

```
" curl http://127.0.0.1:8000/api/v1/companies "
```

This returns the companies available through the API.

Get a company

Example using TCS:

```
" curl http://127.0.0.1:8000/api/v1/companies/TCS "
```

Get profit and loss information

```
" curl http://127.0.0.1:8000/api/v1/companies/TCS/pl "
```

Get balance-sheet information

```
" curl http://127.0.0.1:8000/api/v1/companies/TCS/bs "
```

Get cash-flow information

```
" curl http://127.0.0.1:8000/api/v1/companies/TCS/cashflow "
```

Get financial ratios

```
" curl http://127.0.0.1:8000/api/v1/companies/TCS/ratios "
```

Get company tearsheet information

```
"curl http://127.0.0.1:8000/api/v1/companies/TCS/tearsheet "
```

Get sectors

```
" curl http://127.0.0.1:8000/api/v1/sectors "
```

Get companies in a sector

For Information Technology:

```
" curl http://127.0.0.1:8000/api/v1/sectors/Information%20Technology/companies "
```

Get screener results

```
" curl http://127.0.0.1:8000/api/v1/screener "
```

The exact query parameters can be viewed in the Swagger/OpenAPI documentation.

Get portfolio statistics

```
" curl http://127.0.0.1:8000/api/v1/portfolio/stats "
```

These examples provide a simple way to verify that the API is running and returning data.

18. Swagger / OpenAPI Documentation

FastAPI automatically provides interactive API documentation.

After starting the API, open : <http://127.0.0.1:8000/docs>

The Swagger interface allows the analyst to:

1. View available endpoints
2. Expand an endpoint
3. Review parameters
4. Execute a request
5. View the response
6. Understand the API structure

The OpenAPI specification is also stored in : **docs/openapi.json**

This file can be used by tools that support OpenAPI specifications.

19. Cash Flow Intelligence

Cash-flow intelligence provides additional information about the quality and behaviour of company cash flows. The project analyses:

- Operating cash flow
- Investing cash flow

- Financing cash flow
- Cash-flow relationships
- CFO-related indicators
- Capital expenditure intensity
- Distress and deleveraging indicators

The main output is: **output/cashflow_intelligence.xlsx**

Potential distress-related results are stored in: **output/distress_alerts.csv**

The analyst should use these outputs together with the underlying financial statements rather than relying on a single indicator.

20. NLP Pros and Cons

The project uses an automated NLP-based process to generate company-level pros and cons.

The generated output is: **output/pros_cons_generated.csv**

The generator uses predefined financial rules and confidence scores to identify relevant positive and negative observations.

The generated pros and cons are also used in company tearsheets.

The analyst should understand that these statements are generated from project data and rules. They should therefore be interpreted as analytical summaries rather than independent external research.

21. Company Clustering

The project groups companies using KMeans clustering. The clustering process uses financial characteristics such as:

- ROE
- Debt-to-equity
- Revenue CAGR
- FCF CAGR
- Operating margin

Five labelled company archetypes are used:

- Diversified Core Companies
- Growth & High-Margin Leaders
- Leveraged Financials
- High-Margin Defensive
- Strategic Industrials

The cluster assignments are stored in : **output/cluster_labels.csv**

Additional cluster analysis files include:

output/cluster_profile.csv

output/outlier_report.csv

output/portfolio_stats.csv

reports/elbow_plot.png

reports/correlation_heatmap.png

Clustering is intended to provide an additional analytical view of the companies based on their financial characteristics.

22. Valuation Analysis

The project contains valuation-related outputs that help identify companies based on available valuation metrics.

The main valuation output is : **output/valuation_summary.xlsx**

Valuation flags are stored in : **output/valuation_flags.csv**

The analyst can use valuation information together with profitability, growth, leverage and cash-flow information. Valuation metrics should always be considered in context because a single valuation ratio does not provide a complete view of a company.

23. Important Output Files

The following files are useful when reviewing the project:

File	Purpose
output/analysis_parsed.csv	Parsed financial analysis metrics
output/capital_allocation.csv	Capital allocation analysis
output/cashflow_intelligence.xlsx	Cash-flow intelligence
output/cluster_labels.csv	Company cluster assignments

output/distress_alerts.csv	Distress-related alerts
output/outlier_report.csv	Cluster/outlier analysis
output/pattern_changes.csv	Capital allocation pattern changes
output/peer_comparison.xlsx	Peer comparison output
output/pros_cons_generated.csv	Generated company pros and cons
output/screener_output.xlsx	Screener output
output/valuation_summary.xlsx	Valuation summary
output/valuation_flags.csv	Valuation flags
output/validation_failures.csv	Data-quality validation failures

24. Common Troubleshooting

Problem 1 — Streamlit command is not recognised

Cause: The virtual environment may not be activated or Streamlit may not be installed.

Solution:

Activate the virtual environment:

```
.\venv\Scripts\Activate.ps1
```

Then verify Streamlit:

```
python -m streamlit --version
```

Problem 2 — Dashboard cannot import project modules

Cause: The src directory may not be available in the Python path.

Solution:

Run:

```
$env:PYTHONPATH="$PWD\src"
```

Then start Streamlit again:

```
streamlit run src/dashboard/app.py --server.port 8501
```

Problem 3 — Port 8501 is already in use

Cause: Another Streamlit process may already be running.

Solution:

Either stop the existing process or use another port.

Example:

streamlit run src/dashboard/app.py --server.port 8502

Then open the corresponding port in the browser.

Problem 4 — FastAPI does not start

Cause: The Python environment may not be activated, dependencies may be missing, or the API module may not be started from the correct project location.

Solution:

Check the virtual environment and confirm that the API dependencies are installed.

Run the API from the project root and verify the application module.

After starting the API, check:

<http://127.0.0.1:8000/api/v1/health>

Problem 5 — API health check fails

Cause: The API server may not be running or the requested port may be incorrect.

Solution:

Start the FastAPI application and then test:

curl <http://127.0.0.1:8000/api/v1/health>

If the health endpoint responds successfully, the API is available.

Problem 6 — Database not found

Cause: The application cannot find the SQLite database.

Solution:

Confirm that the following file exists: **data/nifty100.db**

Also make sure commands are being executed from the project root.

Problem 7 — Company data is missing

Cause: Some companies may not have complete data for every financial year.

Solution:

Check the underlying database and data-quality outputs before assuming that the dashboard or API is incorrect.

The validation file is : **output/validation_failures.csv**

The project also handles unavailable values in the dashboard by displaying them appropriately rather than treating missing information as a valid financial value.

Problem 8 — PDF tearsheet has a layout problem

Cause: A long company name, financial value or generated text may require additional wrapping.

Solution:

Check the generated PDF under:

reports/tearsheets/

Review the company name, KPI cards, charts, pros and cons and cash-flow sections.

The tearsheet generator is located at: **src/reports/tearsheet.py**

Problem 9 — Tests fail

Cause: A dependency, environment setting or implementation change may affect the test suite.

Solution:

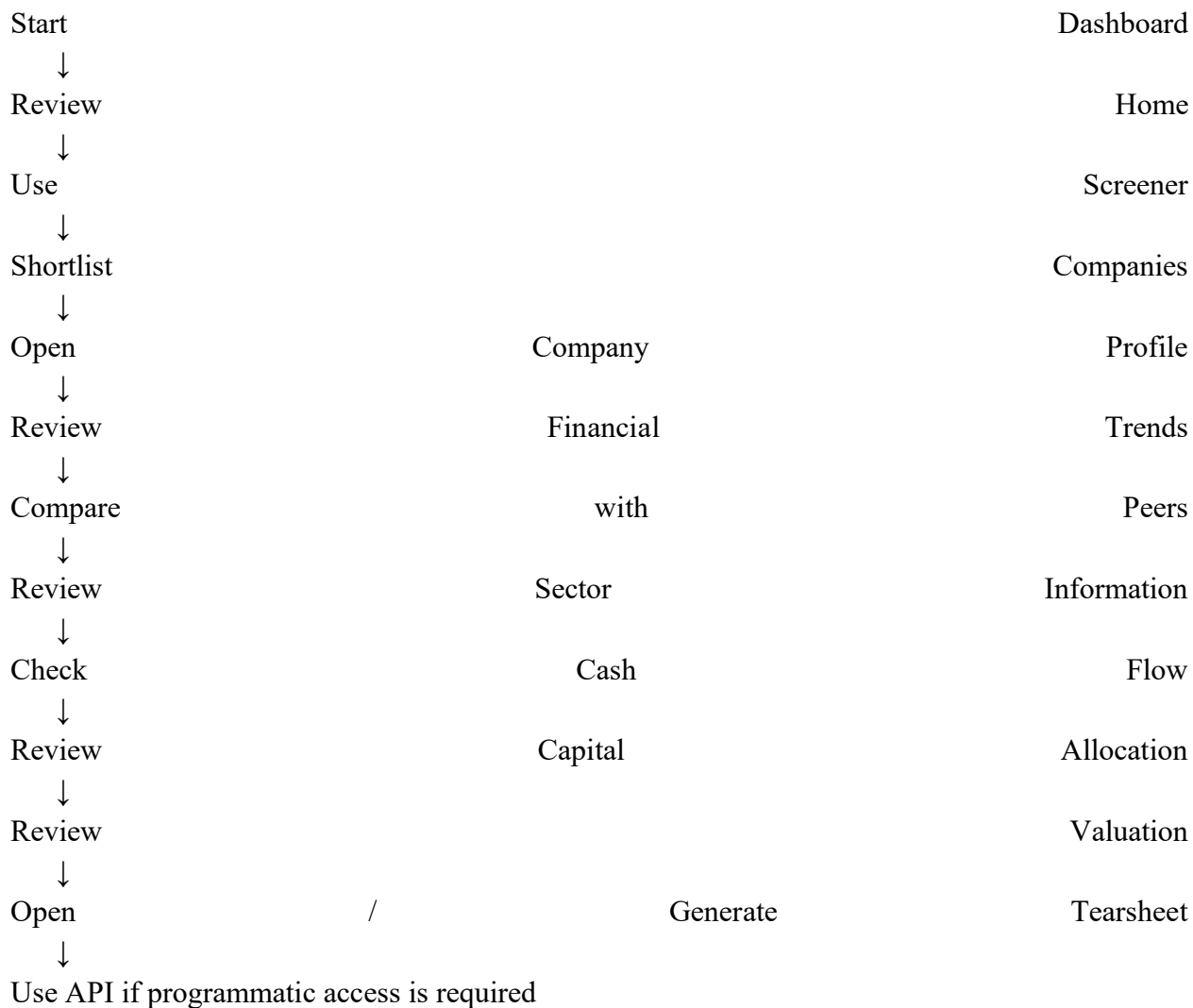
Run the test suite from the project root: **pytest tests\ -v**

Review the failure message and identify whether the issue is related to the environment, data or implementation.

The generated test report is available at: [reports/pytest_report.html](#)

25. Recommended Analyst Workflow

A typical analyst workflow can follow these steps:



This workflow allows the analyst to move from broad screening to detailed company-level analysis.

The dashboard, API and PDF reports use the project's structured data and generated analysis outputs, allowing the analyst to access the same information through different interfaces.

26. API and Dashboard Quick Reference

Streamlit

Start command:

```
$env:PYTHONPATH="$PWD\src"  
streamlit run src/dashboard/app.py --server.port 8501
```

Dashboard:

" <http://127.0.0.1:8501> "

FastAPI

API base URL:

" <http://127.0.0.1:8000> "

Health endpoint:

" <http://127.0.0.1:8000/api/v1/health> "

Swagger:

" <http://127.0.0.1:8000/docs> "

OpenAPI specification:

" docs/openapi.json "

Reports

Company tearsheets: **reports/tearsheets/**

Sector reports: **reports/sector/**

Portfolio report: **reports/portfolio/**

27. Data Quality and Interpretation

The project includes validation and quality-control processes to identify issues in the underlying financial data. The analyst should review data availability before drawing conclusions, especially when comparing companies with different historical coverage.

Missing values should not automatically be interpreted as zero. Where data is unavailable, the dashboard and reports may display an appropriate missing-value indicator.

Historical financial metrics should be considered over multiple years where possible.

Ratios and valuation metrics should also be interpreted in the context of the company's sector, business model and financial history.

The project provides analytical tools to support financial review. The outputs should be treated as structured analysis of the available project data.

28. Final Reference

The Nifty 100 Data Foundation provides three main ways to work with the project:

Interactive Dashboard

Use Streamlit when visual exploration, filtering and company comparison are required.

REST API

Use FastAPI when financial information needs to be accessed programmatically or tested through API tools.

PDF Reports

Use company tearsheets, sector reports and portfolio reports when a fixed report format is required for review or sharing.

The recommended approach is to use these interfaces together. The dashboard is suitable for interactive exploration, the API is suitable for programmatic access, and the reports are suitable for structured documentation and review.

This Analyst Guide provides the basic steps required to start the project, navigate the dashboard, use the screener, access the API, generate reports and troubleshoot common issues.